Guia dos testes de treinamento — Track C

Projeto Pestevision • Documento em português • 14/10/2026

1. Objetivo e alcance

Este guia explica os nove testes de `tests/test_training.py`: como foram construídos, quais comportamentos verificam, como executá-los e como interpretar os resultados. Eles verificam a estrutura de treinamento e avaliação solicitada na Track C sem depender do dataset real, das arquiteturas finais ou das estratégias de aumento de dados.

Os testes usam CPU, tensores pequenos e arquivos temporários. Não é necessário baixar as imagens do projeto nem ter GPU. Passar nesses testes demonstra os comportamentos cobertos; não demonstra qualidade de classificação de pragas, convergência em dados reais ou reprodução integral do artigo. A aderência exata à seção 3.6 ainda precisa ser conferida com o artigo.

Situação da validação: na revisão anterior, a sintaxe foi verificada, mas a suíte não executou porque faltava PyTorch na `.venv`. A instalação temporária também falhou por timeout de rede. Os resultados abaixo são expectativas definidas pelas asserções, não resultados de uma execução bem-sucedida já observada.

2. Como os testes foram construídos

A classe `TrainingTests` herda de `unittest.TestCase`, da biblioteca padrão do Python. Cada método cujo nome começa com `test_` é descoberto como um teste independente. Não é necessário instalar pytest.

Cada caso prepara uma entrada pequena, executa uma função e compara o resultado com uma expectativa conhecida. Matrizes de confusão e sequências de perdas escolhidas manualmente permitem prever o resultado sem depender de um treinamento longo ou aleatório.

- `assertEqual`: compara valores exatos, como o número de épocas.
- `assertAlmostEqual`: compara métricas decimais com tolerância a arredondamentos.
- `torch.testing.assert_close`: compara tensores, como os pesos restaurados.
- `assertRaises(ValueError)`: confirma que entradas inválidas são rejeitadas. A exceção esperada faz o teste passar.
- `subTest`: identifica qual entrada falhou dentro de um mesmo método; não aumenta a contagem dos nove testes.
- `patch`: substitui temporariamente `pestevision.train.run_epoch` por uma função controlada em dois testes. Assim, testa scheduler, parada e checkpoint sem depender da evolução real de uma rede.
- `TemporaryDirectory`: isola os arquivos de checkpoint e histórico e os remove ao terminar o contexto.

Os testes com `patch` não comprovam o cálculo de gradientes. Por isso existe também um caso que executa `run_epoch` de verdade com Adam e verifica a mudança dos pesos.

3. Explicação de cada teste

3.1. Métricas com classes desbalanceadas e ausentes

Método: `test_metrics_imbalanced_and_missing_class`.

Usa a matriz abaixo. As linhas representam classes verdadeiras e as colunas representam classes preditas.

```
          Predita 0  Predita 1  Predita 2
Real 0        2          1          0
Real 1        0          1          0
Real 2        0          0          0
```

Há quatro amostras, três acertos e suportes por classe iguais a 3, 1 e 0. O teste inclui uma classe ausente para verificar a convenção de usar zero em divisões indefinidas e incluir todas as classes configuradas na média macro.

Métrica	Resultado esperado
Accuracy	3/4 = 0,75
Precision macro	1/2 = 0,50
Precision weighted	7/8 = 0,875
Recall macro	5/9 ≈ 0,555556
Recall weighted	3/4 = 0,75
F1 macro	22/45 ≈ 0,488889
F1 weighted	23/30 ≈ 0,766667

Precision mede acertos entre as predições de uma classe; recall mede acertos entre as amostras verdadeiras daquela classe. F1 combina precision e recall por classe. Macro é a média simples dos valores por classe; weighted pondera esses valores pelo suporte real. O F1 agregado é calculado a partir dos F1 por classe, não a partir das médias de precision e recall. Accuracy é a fração global de acertos.

Também verifica que uma matriz totalmente zerada gera `ValueError`, pois não há amostras para avaliar.

3.2. Rejeição de matrizes inválidas

Método: `test_invalid_confusion_counts`.

Passa quatro entradas inválidas: contagem fracionária `[[0.5]]`, negativa `[[-1]]`, não finita `[[NaN]]` e matriz não quadrada `[[1, 2]]`. Todas devem gerar `ValueError`. Isso impede métricas calculadas sobre contagens sem significado válido.

3.3. Tolerância do early stopping

Método: `test_stopping_tolerance`.

Configura paciência 2 e `min_delta=0.1`. `update()` retorna `(melhorou, deve_parar)`. Uma melhora precisa ser estritamente maior que a tolerância para redefinir a referência e zerar o contador.

Perda recebida	Retorno esperado	Explicação
1,00	(True, False)	Estabelece a primeira referência.
0,95	(False, False)	Redução de 0,05 é insuficiente.
0,85	(True, False)	Redução de 0,15 em relação a 1,00 reinicia o contador.
0,84	(False, False)	Primeira época sem melhora suficiente.
0,83	(False, True)	Segunda época sem melhora suficiente.

Também exige `ValueError` para `min_delta` negativo, NaN ou infinito. Esse teste verifica o early stopping isoladamente; o scheduler mantém seu próprio critério de melhora.

3.4. Entradas inválidas em uma época

Método: `test_invalid_epoch_inputs`.

Chama `run_epoch` com batch vazio, quantidades diferentes de imagens e rótulos, rótulo 2 quando só existem as classes 0 e 1, e rótulo em ponto flutuante. Cada caso deve gerar `ValueError`. Também verifica rejeição de `num_classes=0`.

O contrato aceito é um batch `(images, labels)`, com rótulos inteiros `int64` unidimensionais e logits do modelo no formato `[quantidade_de_amostras, quantidade_de_classes]`.

3.5. Limite da paciência e reinício

Método: `test_patience_boundaries_and_reset`.

Usa a paciência padrão de 21. A primeira perda 1,0 estabelece o melhor valor; as próximas 20 perdas iguais não interrompem. A 21ª perda consecutiva sem melhora deve sinalizar parada. Em seguida, uma perda 0,5 deve zerar o contador e retornar `(True, False)`.

A última chamada testa a classe isoladamente: em um treinamento normal, `fit` já teria encerrado o loop quando recebeu o sinal de parada.

3.6. Loop, scheduler, parada e restauração do melhor modelo

Método: `test_full_loop_schedule_stop_and_best_restoration`.

Define no máximo 40 épocas, mas substitui `run_epoch` por uma função que sempre retorna perda 1,0. Em cada chamada de treino, soma 1 aos pesos; na validação, não altera os pesos. Isso torna identificável a época cujos pesos foram restaurados.

Resultados esperados:

- O histórico contém 22 épocas: uma primeira época de referência e 21 sem melhora.
- Na época 11 (`history[10]`), `next_lr` ainda é 0,001.
- Na época 12 (`history[11]`), `next_lr` passa a 0,0001.
- A última época registra `early_stopped=True`.
- Os pesos finais são os pesos iniciais mais 1, correspondentes à primeira e melhor época.

A redução ocorre na época 12 porque `ReduceLROnPlateau(patience=10)` reduz quando o número de épocas ruins ultrapassa 10. A primeira época é a referência; as épocas 2 a 12 completam 11 épocas ruins. O checkpoint é salvo e carregado de verdade, mesmo com a época simulada.

3.7. Checkpoint com melhorias menores que a tolerância

Método: `test_best_checkpoint_includes_improvements_below_tolerance`.

Simula perdas de validação 1,00, 0,99 e 0,98, com `min_delta=0.1` e paciência 2. As melhorias são pequenas demais para reiniciar o early stopping, mas ainda representam modelos com menor perda.

Na terceira época, `early_stopped` e `improved` devem ser ambos verdadeiros: a execução para, mas o melhor checkpoint é atualizado. Os pesos restaurados devem ser os iniciais mais 3. O teste protege a distinção entre tolerância para continuar treinando e seleção do menor valor real de perda.

3.8. Avaliação com batches desiguais e sem gradientes

Método: `test_evaluation_uneven_batches_no_grad`.

Cria três amostras e batches de tamanho 2, resultando em um batch com duas amostras e outro com uma. Uma camada linear com matriz identidade preserva os logits escolhidos no teste. As predições são `[0, 1, 0]`, enquanto os rótulos são `[0, 1, 1]`.

Espera accuracy de 2/3 e perda igual à entropia cruzada calculada diretamente sobre as três amostras. Isso detecta o erro de tirar uma média simples das perdas dos batches sem ponderá-las pelo número de amostras.

Também verifica que o gradiente dos pesos continua `None`, o modelo está em modo de avaliação (`model.training=False`) e um loader vazio gera `ValueError`.

3.9. Atualização real de pesos e repetibilidade dos dados dummy

Método: `test_real_gradient_updates_and_dummy_repeatability`.

Cria loaders duas vezes com seed 42 e três classes. As imagens de validação devem ser iguais entre as duas construções; os tensores de imagens de validação e teste devem ser diferentes.

Depois executa uma época real de treinamento com o modelo dummy e Adam. Os pesos da última camada devem mudar. Esse caso verifica que o caminho de treinamento efetivamente realiza atualização dos parâmetros; não exige que a loss diminua nem uma accuracy específica.

A comparação dos tensores não é uma auditoria completa de ausência de vazamento entre datasets reais, e a repetibilidade verificada se refere às imagens dummy de validação.

4. Como executar

Execute os comandos a partir da raiz do repositório, onde fica `pyproject.toml`. É necessário Python compatível com o projeto (3.12 ou superior), `uv` e as dependências instaladas. A primeira instalação requer acesso à rede.

```
cd /home/josue/Pestevision
uv sync
uv run python -m unittest discover -s tests -v
```

`uv sync` prepara o ambiente conforme as dependências e o lock do projeto. O treinamento e os testes podem rodar em CPU, embora a configuração de instalação do projeto use um índice PyTorch CUDA e possa baixar pacotes grandes. Não altere as versões ou o lock apenas para contornar um download interrompido sem avaliar o impacto no projeto.

Se o ambiente já estiver instalado, também é possível usar diretamente seu Python:

```
.venv/bin/python -m unittest discover -s tests -v
```

Para executar somente um caso, use o filtro de nome do unittest:

```
uv run python -m unittest discover -s tests -v -k test_stopping_tolerance
```

Opcionalmente, no Linux, limite threads para esses testes pequenos:

```
OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 uv run python -m unittest discover -s tests -v
```

O `torch.set_num_threads(1)` no final de `test_training.py` só executa quando o arquivo é o programa principal. A descoberta via `unittest discover` importa o módulo e não executa esse bloco.

5. O que deve aparecer

Em uma execução bem-sucedida da versão documentada, os nove métodos aparecem com `ok`, seguidos de um resumo como:

```
Ran 9 tests in <tempo variável>s
```

```
OK
```

Esse trecho é ilustrativo. O tempo varia conforme a máquina. As chamadas a `fit()` também imprimem linhas `epoch=... train_loss=... val_loss=... lr=...`; elas são normais e podem aparecer misturadas ao relatório, pois o runner e os prints usam fluxos diferentes.

Os testes com diretórios temporários não deixam `best.pt` e `history.json` no projeto. Não espere um relatório de accuracy real nem imagens classificadas ao rodar a suíte.

6. Como interpretar problemas

Saída ou sintoma	Significado e ação
`OK`	Todas as asserções executadas passaram. Confira se foram descobertos os nove testes.
`FAILED (failures=...)`	Uma expectativa não foi atendida. Leia o nome do teste e os valores esperado/obtido no traceback.
`FAILED (errors=...)`	O teste não concluiu por uma exceção inesperada, inclusive problemas de importação.
`Ran 0 tests`	Não houve validação. Confira a pasta atual e use `discover -s tests`.
`No module named 'torch'`	Falta PyTorch no Python utilizado. Execute `uv sync` e use `uv run`.
`No module named 'pestevision'`	O pacote local não está disponível nesse ambiente. Execute `uv sync` na raiz e repita com `uv run`.
Timeout durante instalação	A suíte ainda não foi executada. Verifique rede/acesso ao índice e repita a instalação.
Métricas diferentes do esperado	Verifique a orientação da matriz, classes ausentes, suporte e tratamento de divisões por zero.
Scheduler com uma época de diferença	Confira a primeira época de referência e o comportamento de ultrapassar a paciência.

Não troque valores esperados apenas para fazer o teste passar: primeiro determine se houve erro no código, no teste ou uma mudança intencional de requisito.

7. Execução demonstrativa do pipeline completo

Após os testes, pode-se exercitar separadamente a interface de linha de comando:

```
uv run python -m pestevision.train --epochs 3 --output-dir outputs/guia-track-c
uv run python -m pestevision.evaluate outputs/guia-track-c/best.pt --output
outputs/guia-track-c/test.json
```

Use uma pasta de saída nova para cada execução que deseja preservar. Com os providers dummy atuais, espera-se um histórico com três épocas, um `best.pt` da época com menor perda de validação e um `test.json` com loss, accuracy, precision/recall/F1 macro e weighted, matriz de confusão, número de amostras, split e época do checkpoint.

O loader dummy de teste tem 24 amostras. Não há accuracy-alvo: imagens e rótulos são sintéticos. A avaliação usa o melhor checkpoint, que não precisa corresponder à terceira época. Esse exemplo verifica também a interface de linha de comando e os artefatos, que não são integralmente cobertos pela suíte.

8. Limites atuais e manutenção

A suíte não cobre integração com imagens reais, arquiteturas finais, augmentations, GPU, retomada de treinamento ou todos os parâmetros inválidos possíveis. A retomada por checkpoint não está implementada. A seleção dos providers reais ocorre em `src/pestevision/interfaces.py`.

Ao modificar uma regra, acrescente um caso pequeno que diferencie claramente o comportamento correto do incorreto. Mantenha os testes independentes de downloads de datasets e de resultados aleatórios de treinamento. Atualize este guia se mudar nomes, quantidade de testes ou contratos.

Arquivos de referência: `tests/test_training.py`, `src/pestevision/train.py`, `src/pestevision/evaluate.py`, `src/pestevision/dummy.py` e `src/pestevision/interfaces.py`.